

Basketball Game Features and Implementation Logic

1. BP_BasketballManager (Basketball Game Manager)

1.1 Overview

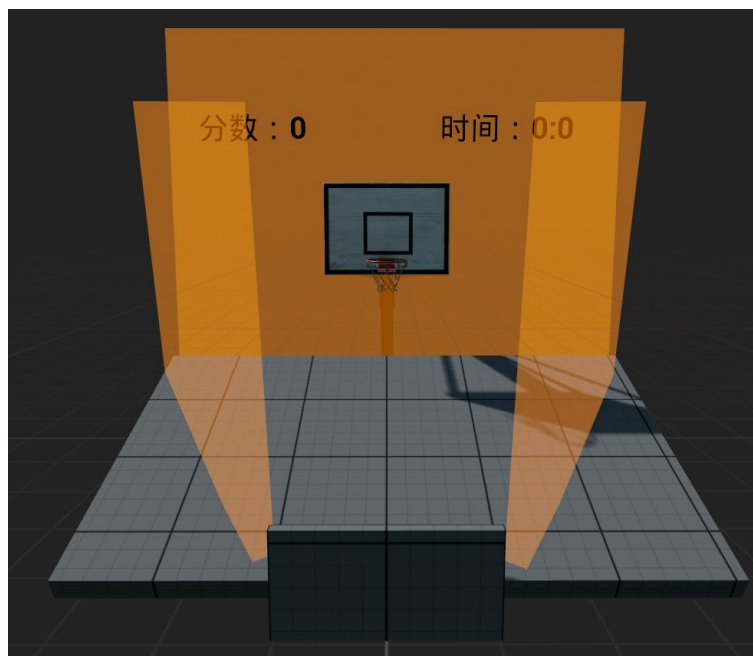
Purpose

BP_BasketballManager is the core manager of the basketball game. It sets up the game area environment, ensures basketballs do not fly out of the play area after being thrown, and allows basketballs to roll back to a pick-up position through physics. The manager also displays the current score and game time.

Description

- Game area setup: fences form a U-shaped space enclosing the basketball hoop and the player's throwing area
- Physical material setup: the basketball uses a physical material so it produces friction and bouncing after landing

Blueprint screenshots



2. BP_Basketball (Basketball Weapon Subclass)

2.1 Overview

Purpose

BP_Basketball inherits from BP_Weapon_Base (weapon base class) and overrides the Shoot and OnHitTarget events to implement a physics-impulse-based (Add Impulse) VR throwing mechanism. After being thrown, the basketball is driven by physics simulation, producing realistic bouncing, rolling, and collisions.

Description

- Shoot event override: applies an instant impulse with Add Impulse, replacing the parent class's projectile movement approach
- OnHitTarget event override: the parent class's default behavior destroys itself; the basketball should not be destroyed, so it is overridden as a no-op
- Physical material setup: the custom physical material PM_Basketball gives the basketball elasticity and friction

2.2 Shoot Event Override

Core principles

- Zeroing velocity: when the basketball is released from a grab, the physics system assigns a release velocity that interferes with the throw direction. Set it to zero before applying the impulse so the ball moves only along the throw direction
- Vel Change: when Add Impulse's Vel Change parameter is true, the impulse magnitude is unaffected by rigid body mass, keeping throw strength consistent across different masses
- Clamp speed limit: the impulse magnitude is clamped to the 800–1200 range, preventing the ball from tunneling through colliders when the player swings too hard, or from failing to reach the hoop when thrown too weakly

Blueprint screenshots

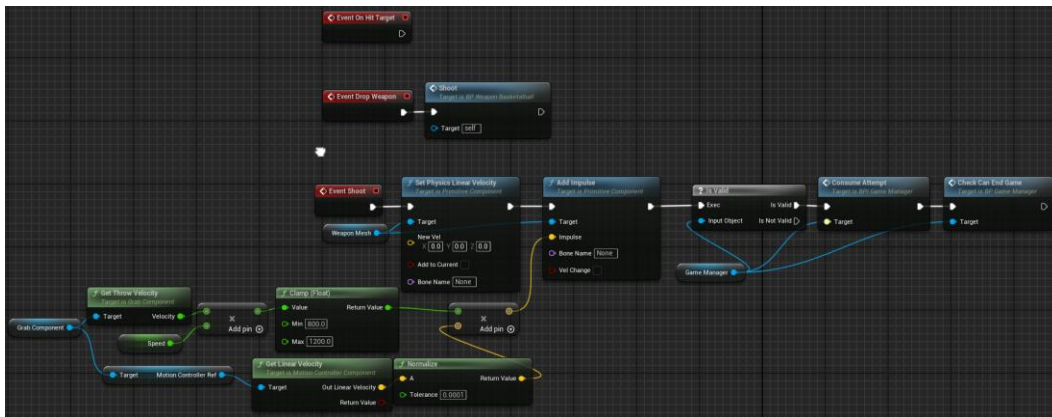


Figure 2-1: BP_Basketball Shoot event Blueprint nodes

2.3 OnHitTarget Event Override

Design

The default behavior of the OnHitTarget event in the BP_Weapon_Base parent class destroys the weapon Actor (suited to one-shot projectiles such as darts and arrows). The basketball is thrown repeatedly and must not be destroyed after hitting a target, so BP_Basketball overrides OnHitTarget as a no-op; after hitting the hoop trigger the basketball is not destroyed and keeps bouncing and rolling in the scene.

2.4 Physics Settings

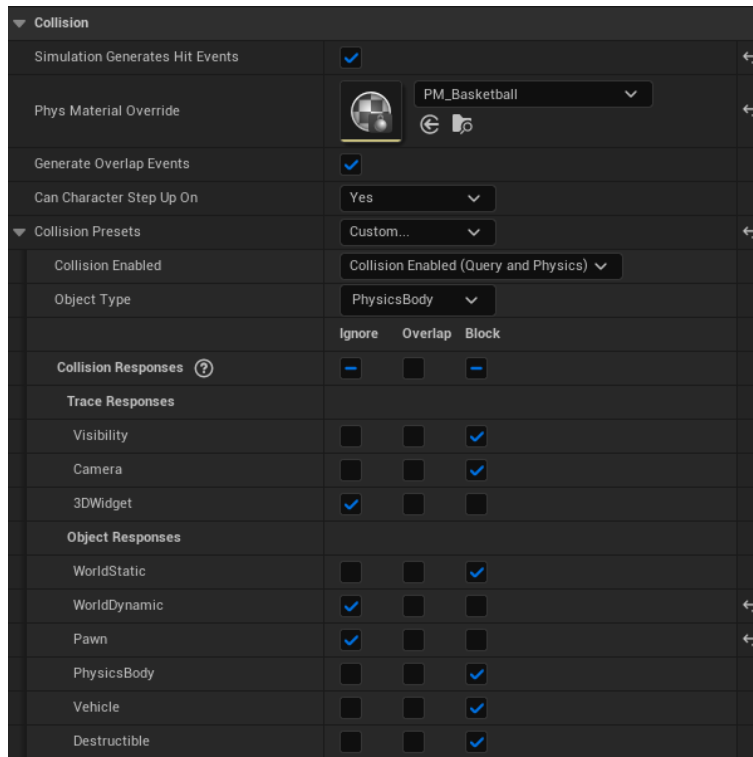
Physical material configuration

Property	Value	Description
Simulate Physics	Enabled	The basketball is affected by gravity, producing falling, bouncing, and rolling
Mass	0.6 kg	Close to a real basketball's mass for authentic physics feel
Linear Damping	0.1	Moderately damps linear velocity so the ball does not roll forever
Angular Damping	0.1	Moderately damps angular velocity so the ball does not spin forever
Phys Material Override	PM_Basketball	Custom physical material controlling elasticity and friction
Restitution (elasticity)	0.6~0.8	High elasticity so the ball visibly bounces after landing
Friction	0.1~0.3	Moderate friction so the ball gradually slows while rolling

Notes

- Simulation Generates Hit Events on the target object is disabled, preventing the OnHit event and the Begin Overlap event from both firing and causing double scoring
- Generate Overlap Events on the trigger stays enabled so the basketball can trigger the Overlap detection

Blueprint screenshots



3. BP_BasketballHoop (Basketball Hoop Target Blueprint)

3.1 Overview

Purpose

BP_BasketballHoop is the basketball hoop target Blueprint, containing the backboard, rim, and net. A trigger (Box Collision) is placed at the rim's center; when the basketball passes through the rim it triggers an Overlap event that runs the scoring logic: validate the colliding object, play a sound, and add points.

Description

- Backboard: a static mesh; when the player's shot misses the rim, the ball can bounce off the backboard into the hoop
- Rim: a static mesh; the basketball must pass through the rim from above to score
- Net: visual effect only, no physical collision
- Trigger (Box Collision): placed at the rim's center to detect the basketball passing through

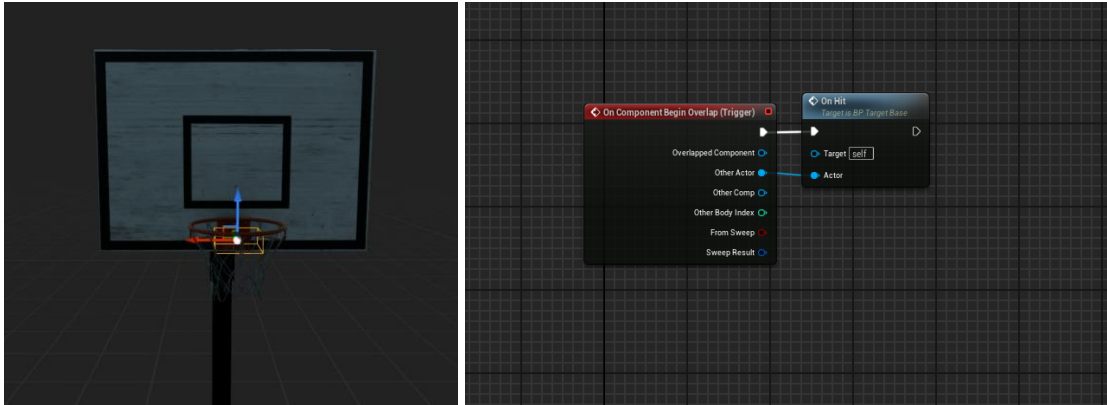
3.2 Score Detection Logic

Design

- A trigger (Overlap) is used instead of the Hit event to detect scoring, avoiding misjudgment from the basketball physically colliding with the rim

- Cast To BP_Weapon_Base validates the colliding object's type, ensuring only weapons (the basketball) can trigger scoring and excluding interference from other Actors
- Simulation Generates Hit Events on the meshes is disabled so only the Overlap event triggers scoring, avoiding double scoring

Blueprint screenshots



Notes

- The trigger should sit at the center directly below the rim so it fires only when the ball passes through from above; side contact does not misjudge
- The trigger should not be too large, or a ball grazing the rim's edge would also score; nor too small, or a fast-moving ball might be missed

4. Complete Throwing Flow

Execution flow

The complete flow from the player grabbing the basketball to scoring:

1. The player's VR controller approaches the basketball, the Grab Component triggers a grab, and the ball attaches to the controller
2. The basketball's collision is set to Query Only (No Physics), preventing ground reaction forces from lifting the player
3. The player makes a throwing motion and releases the trigger, firing the Shoot event
4. The basketball's linear velocity is zeroed, the throw direction and impulse are computed, and Add Impulse is applied
6. The basketball flies under physics simulation, falls under gravity, and bounces off the fences and the ground
7. If the basketball passes through the rim trigger, BP_BasketballHoop runs the scoring logic: Cast validation → play sound → AddScore(1)
8. If the shot misses, the ball bounces and rolls inside the fences, and the player can pick it up and throw again

9. The GameManager consumes one attempt and checks whether the game end condition is met